Week 5: Database Design — Normalization, Schema Design, ER Modeling

Phase 2: PostgreSQL Mastery | Week 5 of 12

Week Overview

This week focuses on the design skills that separate architects from developers. You will learn normalization forms, understand common design patterns (audit tables, soft deletes, hierarchies, polymorphism), and build complete ER diagrams. By Friday you will be able to design any real-world database schema from requirements alone.

Focus: Good schema design prevents years of technical debt. Learn to design correctly the first time.

Learning Objectives

By the end of this week, you will be able to:

- Explain 1NF, 2NF, 3NF, and BCNF with examples.
- Identify and resolve update, insert, and delete anomalies.
- Distinguish when to denormalize intentionally.
- Design ER diagrams for new systems from requirements.
- Implement common patterns: audit/history, soft delete, polymorphic associations, hierarchies.
- Apply naming conventions used at top tech companies.
- Model many-to-many and self-referencing relationships.
- Evaluate trade-offs: wide vs. narrow tables, inheritance vs. polymorphism.

Required Reading

- `06_Database_Design/` — All files

Daily Schedule

Monday — Normalization Theory (60 min)

Topics:

- First Normal Form (1NF): atomic values, no repeating groups
- Second Normal Form (2NF): no partial dependencies on composite key
- Third Normal Form (3NF): no transitive dependencies
- Boyce-Codd Normal Form (BCNF)
- Anomalies: insert, update, delete anomaly
- When NOT to normalize (read performance, event sourcing)

Exercises:

```
-- BAD: violates 1NF (multiple values in one column)
CREATE TABLE bad_students (
```

```
    student_id INTEGER PRIMARY KEY,
    name       VARCHAR,
    courses    VARCHAR  -- "Math, English, History" -- WRONG!
);

-- GOOD: normalized to 3 tables
CREATE TABLE students (student_id SERIAL PRIMARY KEY, name VARCHAR(200));
CREATE TABLE courses  (course_id  SERIAL PRIMARY KEY, name VARCHAR(200));
CREATE TABLE enrollments (
    student_id INTEGER REFERENCES students(student_id),
    course_id  INTEGER REFERENCES courses(course_id),
    PRIMARY KEY (student_id, course_id)
);

-- Identify transitive dependency (2NF violation example)
-- order_items(order_id, product_id, quantity, product_name, product_price)
-- product_name and product_price depend on product_id, not (order_id + product_id)
-- Fix: move product_name and product_price to a products table
```

Tuesday — Design Patterns (90 min)

Topics:

- Audit/History tables for change tracking
- Soft deletes: `deleted_at` pattern
- Hierarchies: adjacency list vs. closure table
- Polymorphic associations (comments on posts AND videos)
- EAV (Entity-Attribute-Value) vs. JSONB
- Temporal data: SCD Type 1, 2, 3

```
-- Audit table pattern
CREATE TABLE products (
    product_id  SERIAL PRIMARY KEY,
    name        VARCHAR(200),
    price       NUMERIC(10,2),
    updated_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE products_audit (
    audit_id    BIGSERIAL PRIMARY KEY,
    product_id  INTEGER,
    operation   VARCHAR(10),  -- INSERT, UPDATE, DELETE
    old_data    JSONB,
    new_data    JSONB,
    changed_by  VARCHAR(100) DEFAULT current_user,
    changed_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Soft delete pattern
CREATE TABLE users_with_softdelete (
    user_id    SERIAL PRIMARY KEY,
```

```
    email      VARCHAR(300) UNIQUE,
    deleted_at TIMESTAMPTZ
);
-- Partial unique index to allow reuse of email after soft delete:
CREATE UNIQUE INDEX idx_users_email_active
    ON users_with_softdelete (email)
    WHERE deleted_at IS NULL;

-- Closure table for hierarchy (better than adjacency list for queries)
CREATE TABLE categories (id SERIAL PRIMARY KEY, name VARCHAR(200));
CREATE TABLE category_closure (
    ancestor_id   INTEGER REFERENCES categories(id),
    descendant_id INTEGER REFERENCES categories(id),
    depth         INTEGER NOT NULL,
    PRIMARY KEY (ancestor_id, descendant_id)
);

-- SCD Type 2: keep full history of dimension changes
CREATE TABLE customers_scd2 (
    surrogate_key  BIGSERIAL PRIMARY KEY,
    natural_key    INTEGER NOT NULL,
    name           VARCHAR(200),
    email          VARCHAR(300),
    effective_from TIMESTAMPTZ NOT NULL,
    effective_to   TIMESTAMPTZ,  -- NULL = current record
    is_current     BOOLEAN NOT NULL DEFAULT TRUE
);
```

Wednesday — ER Modeling Practice (90 min)

Topics:

- Entities, attributes, relationships
- Cardinality: 1:1, 1:N, M:N
- Crow's foot notation
- Translating ER diagrams to SQL tables
- Design review checklist

Design Exercise: From these requirements, build a complete schema:

```
Requirements: Hospital Management System
- Patients have name, DOB, blood type, insurance info
- Doctors have name, specialization, license number
- Departments have name and head doctor
- Appointments link a patient to a doctor with date/time and status
- Each appointment can have multiple diagnoses (ICD codes)
- Prescriptions are written at an appointment for specific medications
- Medications have name, manufacturer, dosage forms
- Rooms have a number, floor, type (ICU, General, Surgery), and capacity
- Patients are admitted to rooms with admission and discharge dates
```

```
-- Write the complete CREATE TABLE statements for the hospital system
-- Include all foreign keys, CHECK constraints, and indexes
-- Aim for at least 8 tables

-- After writing, evaluate against:
-- [ ] No repeating groups
-- [ ] No partial dependencies
-- [ ] No transitive dependencies
-- [ ] All relationships properly cardinality-constrained
-- [ ] Meaningful naming conventions
-- [ ] All necessary indexes planned
```

Thursday — Naming Conventions and Standards (60 min)

Topics:

- Snake_case vs. CamelCase (PostgreSQL lowercase)
- Singular vs. plural table names (pick one, stick to it)
- ID column naming: `id`, `user_id`, `users_id`
- Timestamp columns: `created_at`, `updated_at`, `deleted_at`
- Junction table naming: `user_roles`, `post_tags`
- Boolean naming: `is_active`, `has_permission`
- Schema namespacing for multi-module systems

```
-- Industry-standard naming example
CREATE TABLE user_accounts (          -- plural, snake_case
    user_account_id SERIAL PRIMARY KEY,
    username        VARCHAR(50) NOT NULL UNIQUE,
    email           VARCHAR(300) NOT NULL UNIQUE,
    password_hash   VARCHAR(256) NOT NULL,
    is_active       BOOLEAN NOT NULL DEFAULT TRUE,
    is_email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    deleted_at      TIMESTAMPTZ  -- soft delete
);

-- Junction table naming
CREATE TABLE user_roles (
    user_account_id INTEGER REFERENCES user_accounts(user_account_id),
    role_id         INTEGER,
    assigned_at     TIMESTAMPTZ DEFAULT NOW(),
    assigned_by     INTEGER REFERENCES user_accounts(user_account_id),
    PRIMARY KEY (user_account_id, role_id)
);
```

Friday — Schema Review + Design Project (45 min)

Mini-Project: Design a social media platform schema.

```
Requirements:
- Users can post text, images, and videos
- Posts can be liked, commented on, and shared
- Users follow other users (directed graph)
- Posts have hashtags
- Users can create groups and post in groups
- Private messaging between users
- Notifications for follows, likes, comments

Design goals:
1. Normalize to 3NF
2. Handle polymorphism for post types
3. Consider query patterns for "feed" generation
4. Plan indexes for the 3 most expensive queries
```

Practice Tasks

1. Normalize a denormalized sales spreadsheet into 3NF (given as a wide table).
2. Implement a self-referencing manager hierarchy with closure table queries.
3. Design a polymorphic comments system (comments on multiple entity types).
4. Implement SCD Type 2 for a customer address that changes over time.
5. Write a trigger-based audit function that captures all changes as JSONB.
6. Design a multi-tenant schema where each tenant's data is isolated.
7. Identify 5 design mistakes in a poorly designed schema and propose fixes.
8. Create the hospital schema from the Wednesday exercise and write 5 queries.

Self-Assessment Checklist

- [ ] I can explain 1NF, 2NF, 3NF to a non-technical person
- [ ] I can identify normalization violations in a given schema
- [ ] I know 5 common database design patterns
- [ ] I designed the hospital schema without assistance
- [ ] I understand when to intentionally denormalize
- [ ] I applied consistent naming conventions to all my schemas
- [ ] I can draw an ER diagram for any system given requirements

---

Mock Interview Questions

1. A candidate says "I always denormalize for performance." How would you respond?
2. What is the difference between 2NF and 3NF? Give an example of a 2NF violation.
3. When would you use a closure table instead of an adjacency list for hierarchies?
4. How do you model a polymorphic relationship (e.g., comments on posts AND videos)?
5. Explain the trade-offs of soft delete vs. hard delete.
6. Design a schema for a hotel booking system. Walk me through your decisions.
7. What is an update anomaly? Give an example in a poorly designed schema.
8. How would you implement versioning/history for a frequently updated config table?

Resources

- This repo: `06_Database_Design/`
- "Database Design for Mere Mortals" by Michael Hernandez
- ERDPlus online diagramming: https://erdplus.com